Reasoning and Retrieval for Complex Semi-structured Tables via Reinforced Relational Data Transformation

Haoyu Dong
Institute of Information Engineering, Chinese Academy of Sciences
School of Cyber Security, University of Chinese Academy of Sciences
Beijing, China
donghaoyu@iie.ac.cn

Yue Hu
Institute of Information Engineering, Chinese Academy of Sciences
School of Cyber Security, University of Chinese Academy of Sciences
Beijing, China
huyue@iie.ac.cn

Yanan Cao*
Institute of Information Engineering, Chinese Academy of Sciences
School of Cyber Security, University of Chinese Academy of Sciences
Beijing, China
caoyanan@iie.ac.cn

Abstract

We introduce TabFormer, a framework that normalizes one or multiple semi-structured tables into relational data via large language models to facilitate various table retrieval and reasoning tasks. Our approach employs a chain-of-thought methodology, transforming one or multiple tables through a sequence of soft operations. Compared to existing hard-coded operators, soft operators are designed with greater flexibility to accommodate diverse human-induced artifacts in real-world tables.

To address the lack of ground-truth labels for table transformation, we propose a reinforced fine-tuning strategy that sequentially and jointly optimizes table transformation and symbolic reasoning within a single LLM call. This process is guided by two novel reward functions without requiring human annotations on table transformation: (1) relational normalization quality and (2) symbolic reasoning accuracy.

TabFormer is a scalable, one-time framework that leverages LLMs to transform one or multiple tables in a single inference step, thereby enhancing both retrieval and reasoning for a wide range of tabular data tasks. Experimental evaluations on datasets such as WTQ, HiTab, MultiHiertt, and TabFact demonstrate significant gains in accuracy—improvements ranging from 4% to 17%—when applied to state-of-the-art methods, including GPT-4-based E5, Chain-of-table, TableLlama, and others.

CCS Concepts

• **Information systems** → **Information retrieval**.

Keywords

Information Retrieval; Table Generation; Symbolic Reasoning

ACM Reference Format:
Haoyu Dong, Yue Hu, and Yanan Cao*. 2025. Reasoning and Retrieval for Complex Semi-structured Tables via Reinforced Relational Data Transformation. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '25), July 13–18, 2025, Padua, Italy.* ACM, New York, NY, USA, 10 pages. https://doi.org/10.1145/3726302.3730071

* Corresponding author.

SIGIR '25, Padua, Italy

ACM ISBN 979-8-4007-1592-1/2025/07
https://doi.org/10.1145/3726302.3730071

1 Introduction

The rapid development of Large Language Models (LLMs) has opened new possibilities for data retrieval and analytics through tools like SQL and Python Pandas [10, 30]. However, their effectiveness heavily relies on data being in a relational format. When tables contain merged cells, empty or sparse rows, and hierarchical headers [7, 15, 21]—often with functional, hierarchical, and computational dependencies [8, 9, 14, 29]—they become challenging for machines to process.

While substantial prior work has focused on understanding complex table structures [21, 24, 32, 54, 58, 64, 71], and some studies have explored designing domain-specific languages for handling semi-structured tables [9, 39], a widely recognized interface is still missing in the era of LLMs to enable program-aided table retrieval and analytics. Fortunately, relational tables provide a powerful solution by enabling LLMs to use indexing and query languages. By extracting the relational representation of complex tables, LLMs can leverage powerful data indexing and analysis capabilities using various existing query and analysis languages.

To this end, we introduce TabFormer, a framework designed to transform semi-structured tables into relational data, as illustrated in Figure 1. TabFormer can process data online to support reasoning and analysis for complex tables or proactively handle tables offline for efficient and accurate information retrieval. Unlike existing table transformation approaches that operate on a single input table to produce a single output table [29, 37], we address the more complex problem of handling one or multiple input tables and producing one or multiple output tables. The common use of SQL in databases with complex multi-table dependencies enables LLMs to excel at generating relational tables that are easy to query. Therefore, to promote relational normalization and facilitate subsequent symbolic retrieval and reasoning tasks, we adopt SQL to generate the transformed relational tables.

To preserve source information reliably, TabFormer introduces a *chain-of-transformation* paradigm, where semi-structured tables undergo a sequence of transformation operators—such as unfolding multi-level headers, splitting composite cells, and pivoting row groups—until a properly normalized relational table emerges, as illustrated in Figure 2. To handle the unpredictable artifacts frequently observed in real-world tables (e.g., merged cells, indented hierarchies, or partially repetitive row groups), we design these operators in a *soft* manner. Rather than using hard-coded logic, each operator employs high-level semantic parameters to make the

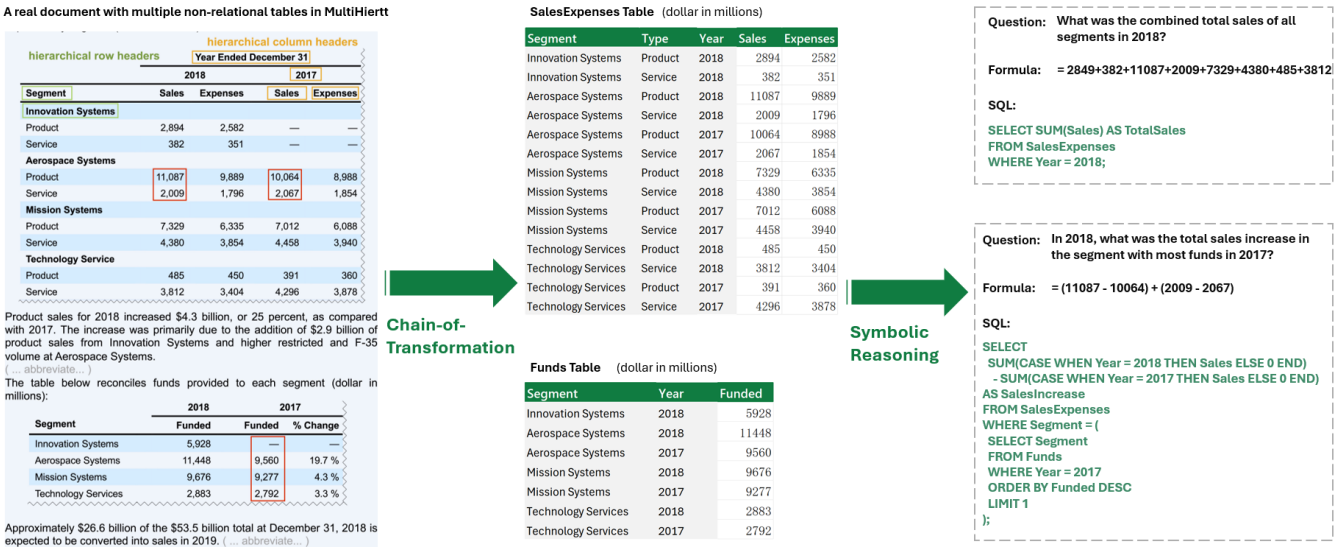

Figure 1: Illustration of relational data transformation. The left side displays a real document from MultiHiertt, while the middle shows the data transformed into a relational format. On the right, two QA examples demonstrate how the relational structure facilitates code(SQL)-based symbolic table reasoning methods [5, 10, 18, 25, 30, 33, 36, 38, 55, 61, 63, 65, 66].

framework more robust to varied table formats, such as the bolded segments in the nested left headers of the first table in Figure 1.

A central challenge lies in the complexity of human annotation for table transformation, making existing supervised fine-tuning (SFT) methods too costly to scale. Moreover, the existence of multiple feasible solutions poses a challenge for SFT, which relies on clear and unambiguous labels. To address this, TabFormer (1) uses SFT to warm up the model on synthetic transformed tables generated through inverse operations on known relational tables; and (2) applies *Reinforcement Learning for LLM Fine-tuning* (ReFT) [31, 31, 35, 40, 49, 52, 59, 62, 70] to explore optimal transformation solutions without relying on table transformation annotations. The ReFT framework optimizes table transformation and symbolic reasoning sequentially and jointly within a single LLM call, guided by two novel reward functions: a relational normalization reward, which evaluates whether the transformed tables are well-normalized, and symbolic reasoning correctness rewards, which assess performance on downstream QA or verification tasks. Our rewards enhance relational normalization and reasoning accuracy, as well as mitigate reward sparsity. A fallback mechanism finally rejects or retries failed transformations.

We validate TabFormer on four tasks—WikiTQ, HiTab, MultiHiertt, and TabFact—covering diverse domains and queries from simple lookups to multi-step reasoning. Across seven strong baselines (including GPT-4-based E5, Chain-of-table, and TableLlama), using TabFormer (Llama-3-8B) for preprocessing yields consistent gains of 4% to 17% in reasoning accuracy. These results highlight the effectiveness of a single, robust normalization step to unlock richer analytics and retrieval on semi-structured tables.

Our contributions are summarized as follows:

- We propose a chain-of-Thought (CoT) approach that employs LLMs to perform step-by-step table transformations in a single inference. To effectively handle the various formatting artifacts present in tables, we design operators for structure transformation and value normalization to be flexible ("soft") by incorporating high-level semantic parameters.
- To the best of our knowledge, our method is the first to propose reinforcement learning for table transformation. TabFormer automatically explored chained table transformation and symbolic reasoning solutions without any human labeling on table transformation. We design a relational normalization reward to encourage normalized outputs, as well as final and stepwise reasoning correctness rewards to enhance reasoning accuracy and mitigate reward sparsity. Finally, a reranking and fallback mechanism is used to mitigate the effect of bad transformations for hard cases.
- TabFormer is a scalable, one-time framework that enhances table reasoning and retrieval across tasks, functioning effectively in both online and offline settings. Evaluations of TabFormer (Llama-3-8B) on datasets such as WTQ, HiTab, MultiHiertt, and TabFact show substantial accuracy gains, ranging from 4% to 17%, compared to strong or SOTA baseline methods, including GPT-4-based E5, Chain-of-table, and TableLlama.

2 Preliminaries

2.1 Table Transformation Task

We propose $T \rightarrow R$ to transform one or multiple complex tables T into one or multiple relational tables R, which significantly enhances the capabilities of existing table reasoning and retrieval

systems designed for relational tables. The transformed data ensures that the original information is preserved while adhering to a relational structure.

2.2 Table Retrieval and Reasoning Tasks

We collect semi-structured tables from diverse sources such as government and financial reports, as well as Wikipedia pages, by leveraging datasets from two widely studied tasks: table QA and fact verification. We adhere to the training/test set splits of all datasets.

2.2.1 Table Question Answering. We use the three most representative datasets for semi-structured tables, HiTab, MultiHiertt, and WikiTQ.

HiTab [9] comprises 3,597 tables collected from government reports and Wikipedia pages, including 10,672 QA pairs. These HTML/spreadsheet tables feature complex structures with multi-level headers and implicit relationships among data entries, making them challenging. HiTab contains 1,584 test samples.

MultiHiertt [69] consists of 10,440 question-answer pairs derived from 2,513 documents sourced from PDF financial reports. This dataset presents complex questions that require retrieving evidence from multiple tables and performing multi-step numerical reasoning. Since the test set has not been made public, we use the validation set for testing, which includes 338 table-only samples.

WikiTQ [39] contains 2,108 HTML tables extracted from Wikipedia, along with 22,033 natural language queries. 20% of the tables and their associated questions are designated for the test set.

2.2.2 Table Fact Verification. TabFact [6] includes 16,573 tables sourced from Wikipedia, paired with 118,275 textual facts. The TabFact test set contains 1,695 unique tables with 12,779 samples.

3 TabFormer

Our method addresses four challenges: (1) How to generate step-by-step transformations; (2) How to encourage relational normalization; (3) How to train a step-by-step transformation solution without labor-intensive, human-labeled data; (4) How to prevent and tackle transformation failures.

3.1 Chain-of-transformation with Soft Operators

AutoTables introduces a pioneering and comprehensive set of table-restructuring operators and parameters to "relationalize" tables [29]. Building on this advantage, we explore extending hard-coded operators, which struggle with the diverse patterns and flexible formats of semi-structured tables, to soft operators. For instance, (1) existing operators are limited to single-table transformations and cannot **split a table into multiple ones**; (2) hard-coded operators are challenged by **merged cells** [9] and other structural and format artifacts such as font bold (as illustrated in nested headers of the first table in Figure 1), indents, or empty rows indicating hierarchy; (3) operators like *explode* or *wide-to-long* depend on a consistent delimiter rather than **underlying semantics**, limiting their ability to parse strings with arbitrary elements like "1983_new-york_city" or "1992: Tokyo and Osaka" [51]; (4) furthermore, **irregular expandable groups** challenge pivot operations that require fixed repeat frequencies [17].

To this end, we propose soft operators with the following new characteristics: (1) operators are executed neurally using LLMs in a chain-of-thought manner, instead of hard-coded systems; (2) operators use high-level semantic parameters to specify header names, rather than relying on rigid indexes. In addition to incorporating operators from AutoTables, we include operators introduced by NormTab [37] for date and numerical value standardization, default value normalization, and composite value splitting (similar to *explode*). Moreover, we introduce a *split* operator to divide a composite table into multiple ones with different primary keys, and we revise the *subtitle* operator to *unfold*, enabling it to manage merged cells in the top header more effectively. Operators are listed in Table 1 and illustrated in the following prompt to LLMs:

Prompt 1: Prompt of TabFormer

```
You are an expert in table transformation and reasoning, equipped
with the following "soft" domain-specific operators. They are...
[documentation of soft operators]

Examples for illustration:
1. unfold:
   - Action: Apply `unfold` to expand these multi-layered headers and
     subtitles into a clearer, more explicit structure, rather than
   merging, subtitling, or flattening them.
   - Target: Hierarchical headers of "Year" and "Sales/Expenses."
2. split:
   - Action: Use `split` to create two distinct table structures,
   each focusing on relevant columns.
   - Target: The columns grouped under "Sales" versus "Expenses."

One or multiple tables in HTML format are given:
[Tables]

Your first goal is table transformation:
Transform the given table(s) one by one into a relational format,
sequentially applying the aforementioned soft domain-specific
operators in a chain-of-thought manner:
- Clearly state each operator you are using.
- For each operator, explain the semantic parameter (which headers
you are targeting).
- Produce the transformed tables for each step in HTML format.

Then construct the final transformed relational tables using SQL
code.

One or multiple questions (or statements) are given:
[Questions (or statements)]

Your second goal is reasoning over the transformed relational tables:
Generate SQL using the question and your transformed relational
table(s) to correctly answer the question (or verify the statement).
```

Leveraging SQL for Relational Table Generation. Generating multiple related tables is crucial for facilitating easy querying and retrieval, yet this has rarely been explored in previous works, which focus solely on single-table generation. There are various strategies to construct relational tables in real practice. For example, Dimensional Modeling, often using Star or Snowflake Schemas, prioritizes query performance and ease of understanding in Online Analytical Processing contexts [26]. It features a more denormalized structure that combines attributes in fact and dimension tables to reduce joins.

The common use of SQL for databases with complex multi-table dependencies helps LLMs excel at generating relational tables that are easy to query. Recent research further shows that encoding tables with SQL boosts QA performance [19, 47, 57]. Therefore,

Table 1: Soft operators comprise both structure transformation [29] and value normalization [37]. High-level semantic parameters have been proposed to manage diverse formatting artifacts.

Operator	Python Pandas related	High-level Semantic parameters	Description
unfold	merged_cells, copy, ffill	a group of headers	propagate multi-level headers to rows/columns
stack	melt	a group of column headers	collapse homogeneous columns into rows
wide-to-long	wide_to_long	a group of column headers	collapse repeating column-groups into rows
pivot	pivot	a group of row headers	pivot repeating row-groups into columns
transpose	transpose	–	convert rows to columns and vice versa
split	array_split, iloc, groupby	at least two groups of headers	split one table into multiple tables
del	drop	a group of headers or cell values	delete rows/columns or cells
explode	explode	a group of headers	convert composite cells into atomic values
dnv-standard	to_datetime, to_numeric	a group of headers	standardize/unify date and numeric values
default-norm	fillna, replace	a group of headers	normalize default values with *NULL*

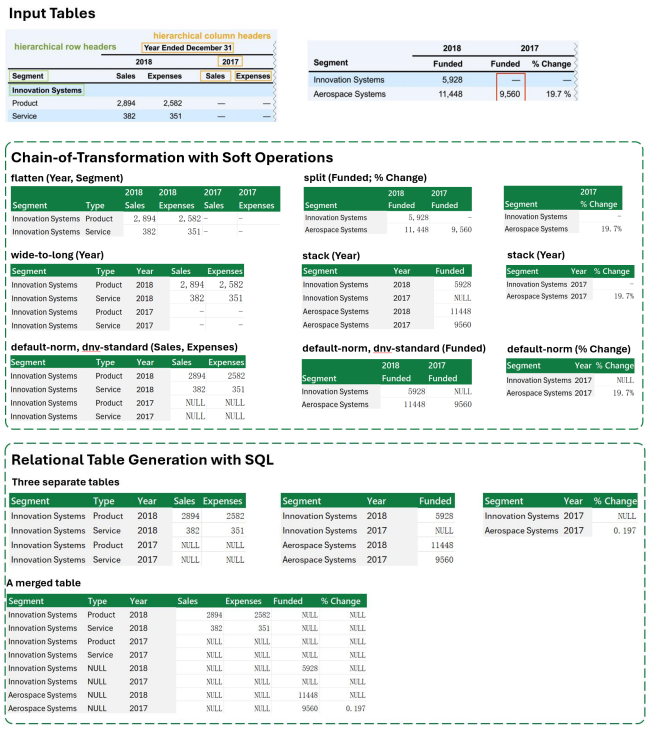

Figure 2: An illustration of Chain-of-transformation.

we use SQL to create relational tables with a comment for the title, though Python pandas is another option. Initially, TabFormer creates CREATE statements to build tables with column headers, primary keys, and type constraints, helping prevent schema and type errors during content generation with INSERT statements. Then SQL code can be generated to join or decompose relational tables to prioritize query performance or minimize redundancy.

```
CREATE TABLE Funded (Segment VARCHAR(50), Year INT, Funded INT,
      PRIMARY KEY (Segment, Year));
```

3.2 Symbolic Information Retrieval and Reasoning for Downstream Tasks

Finally, to bridge the transformed relational tables to symbolic reasoning, we adapt Binder's prompt [11] for question answering and fact verification. Binder generates executable SQL code, requiring tables to have a relational structure for SQL execution. Compared with ReAcTable and Chain-of-table, Binder's prompt integrates more easily without additional environment interactions. SQL code correctness can be evaluated through execution accuracy, used by reasoning rewards in Section 3.3.4. To reduce training costs during ReFT, all questions or statements for the same tables are combined into a single prompt.

3.3 Reinforcement Learning for LLM Fine-tuning

One way to enhance LLMs' reasoning capabilities is through SFT using chain-of-thought annotations. However, in table transformation problems, acquiring human-labeled reasoning paths is too costly to scale. Fortunately, the recently proposed reinforced fine-tuning [49] significantly enhances LLMs' capability for chain-of-thought reasoning through self-exploration of a variety of paths, scalable without relying on human labeling. Additionally, learning from multiple valid paths for the same input improves generalization compared to learning from a single path by SFT.

3.3.1 Algorithm Overview. Our method begins with a warmup phase using SFT and synthetic data, then uses the Proximal Policy Optimization (PPO) algorithm for fine-tuning, where reasoning paths are sampled automatically, guided by two rewards proposed in this paper: relational normalization and reasoning correctness. The policy model learns by sampling responses, evaluating their correctness with reward functions, and updating its parameters online. Following [72], the value model V_ϕ is constructed by adding a linear value head to the last hidden states of the policy model π_θ.

3.3.2 ***Warm-up Fine-tuning.*** The warm-up stage uses SFT to enhance the model's basic ability to generate table transformation responses. While there was no labeled dataset available, fortunately, AutoTables [29] provided a synthesized data framework and dataset

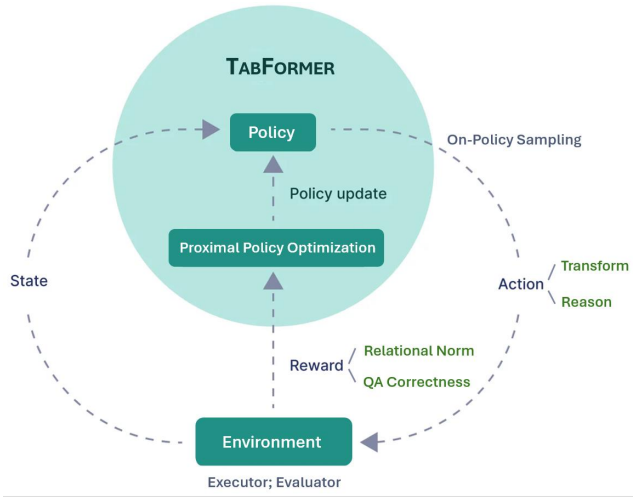

Figure 3: TabFormer's ReFT Algorithm Overview.

for table transformation, which can be leveraged for warm-up fine-tuning. Triples consisting of a Relational table R, a sequence of Operators Os, and a Transformed table T are generated by applying inverse operations to relational tables without manual labeling. We collected over 15,000 relational tables from Power-BI data models and created around 20 augmented tables for each through data augmentation. While synthesized data scales well and ensures accurate transformation results for each operation, the transformed tables often lack realistic formatting artifacts.

We encode (T, Os, R) triples following the structure outlined in Prompt 1 and the operators listed in Table 1, instructing the LLM to provide responses. Formally, the response $\boldsymbol{e}$ is defined as $[a_1, a_2, \ldots, a_{L-1}, a_L = \texttt{<eos>}]$, where L is the maximum length. For each time step t from 1 to $L-1$, the action a_t is sampled based on a policy $\boldsymbol{\pi\theta}(\cdot \mid s_t)$, where a_t can be any token from the vocabulary, and s_t includes all previous tokens. The new state, s_{t+1}, is formed by appending a_t to s_t. The loss function for each sample is formally described by the following equation:

$$\mathcal{L}_{SFT}(\boldsymbol{\theta}) = -\mathbb{E}\boldsymbol{e} \sim \mathcal{D}\left[\sum_{t=1}^{L} \log\left(\boldsymbol{\pi_\theta}(a_t|s_t)\right)\right] \tag{1}$$

The training process starts with single-step table transformations and gradually expands to multi-step ones.

3.3.3 ***Relational Normalization Reward.*** We define r_{norm} to encourage the model to produce normalized relational tables to enable subsequent symbolic reasoning. Although many existing approaches are grounded in data-driven methods [3, 54] or database theory [4, 13, 27], we directly prompt GPT-4o as a judge to score the extent to which generated tables are in a relational form that effectively facilitates symbolic reasoning and retrieval, using a scale from 0 to 1.

$$r_{\text{norm}}(s_t, a_t, s_{t+1}) = LLM\_judge(s_{t+1}) \tag{2}$$

3.3.4 ***Reasoning Correctness Reward.*** We propose two types of reasoning reward functions: a final reward and a stepwise reward.

Final reasoning reward. Let m denote the number of question-answering (QA) and Fact Verification (FV) examples associated with the common input tables. For SQL statements extracted from the final output after reasoning, we evaluate them with Execution Accuracy (EA) as a kind of execution feedback [20] and write the final reasoning reward function as follows:

$$r_{final\_reason}(s_t, a_t, s_{t+1}) = \max\left(0,\ \frac{1}{m}\sum_{i=1}^{m}(\text{EA}_i(s_{t+1}))\right) \tag{3}$$

where $\text{EA}_i(s_{t+1})$ represents the EA score for the i-th QA/FV example at $t+1$. Both the normalization reward and the final reasoning reward are sparse, depending on all previous steps and appearing at the end of transformation and reasoning, respectively.

Stepwise reasoning reward. To mitigate the effect of learning from a sparse final reward [43, 48] and ensure transformed tables at each step are comprehensible to generic LLMs, we define a reward function based on the Reasoning Score (RS) change between tables before and after a transformation. Tables before and after each step are parsed using rules that identify the latest integral table from s_{t+1} and s_t via HTML tags. We then employ three LLMs (Llama-3-8B, Qwen-2.5-7B, Phi-3-mini) to answer queries about tables before and after each transformation. The LLM inference temperature is set to 0 to reduce randomness, although some randomness can aid diverse exploration. RS is calculated as the average exact match score. We denote $\text{RS}_i(s_{t+1})$ and $\text{RS}_i(s_t)$ as the RS scores for the i-th reasoning example at $t+1$ and t, respectively. For each t at the end of a transformation, we write:

$$r_{step\_reason}(s_t, a_t, s_{t+1}) = \max\left(0,\ \frac{1}{m}\sum_{i=1}^{m}(\text{RS}_i(s_{t+1}) - \text{RS}_i(s_t))\right) \tag{4}$$

The model receives positive reinforcement only when the current transformation step results in an improvement in reasoning performance. By setting negative differences to 0, we discourage actions that do not contribute to or detract from reasoning accuracy.

The total reward is the sum of the reward function scores and the Kullback-Leibler (KL) divergence [28] between the learned RL policy and the initial policy, scaled by a coefficient factor β:

$$\begin{aligned} r_{total}(s_t,a_t, s_{t+1}) = r_{norm}(s_t, a_t, s_{t+1}) + r_{final\_reason}(s_t, a_t, s_{t+1}) \\ + r_{step\_reason}(s_t, a_t, s_{t+1}) - \beta KL\left(\boldsymbol{\pi_\theta}(\cdot|s_t), \boldsymbol{\pi}_{\boldsymbol{\theta}}^{(0)}(\cdot|s_t)\right) \end{aligned} \tag{5}$$

Note that the weight of the step reasoning reward gradually diminishes as the number of training iterations increases.

3.3.5 ***Sampling and Optimization.*** We merge the training data from HiTab, MultiHiertt, WikiTQ, and TabFact for training, sample 20% of the tables from TabFact's training set, and construct a training set of 8.1k tables with 46.5k reasoning samples from diverse sources. Utilizing PPO, ReFT can sample multiple correct reasoning paths and learn from them. For advantage calculation, we employ the generalized advantage estimate [44]:

$$\hat{A}_t = \sum_{l=0}^{L-t} (\gamma\lambda)^l (-V_\phi(s_{t+l}) + r_{total}(s_{t+l}, a_{t+l}, s_{t+l+1}) + \gamma V_\phi(s_{t+l+1})) \tag{6}$$

with the terminal state value $V_\phi(s_{L+1}) := 0$, $\lambda \in (0, 1]$ is the discount factor for rewards, and $\gamma \in [0, 1]$ is the discount factor for Temporal Difference (TD). Finally, the loss function is the weighted sum of the policy and value objectives, expressed in the two equations below, with the coefficient α for the value objective:

$$\mathcal{L}_{policy}(\boldsymbol{\theta}) = -\mathbb{E}_{\sim \boldsymbol{\pi}_{\theta_{\text{old}}}} \left[\min \left(\frac{\boldsymbol{\pi}_{\boldsymbol{\theta}}(a_t|s_t)}{\boldsymbol{\pi}_{\theta_{\text{old}}}(a_t|s_t)} \hat{A}_t, \right. \right. \\ \left. \left. \text{clip} \left(\frac{\boldsymbol{\pi}_{\boldsymbol{\theta}}(a_t|s_t)}{\boldsymbol{\pi}_{\theta_{\text{old}}}(a_t|s_t)}, 1-\epsilon, 1+\epsilon \right) \hat{A}_t \right) \right] \tag{7}$$

$$\mathcal{L}_{value}(\boldsymbol{\phi}) = \frac{1}{2} \mathbb{E}_{\sim \boldsymbol{\pi}_{\theta_{\text{old}}}} \left[\max \left(\left\| V_{\boldsymbol{\phi}}(s_t) - (\hat{A}_t + V_{\boldsymbol{\phi}}(s_t)) \right\|^2, \right. \right. \\ \left. \left. \left\| \text{clip} \left(\hat{A}_t, \hat{A}_t - \epsilon, \hat{A}_t + \epsilon \right) \right\|^2 \right) \right] \tag{8}$$

where $\boldsymbol{\pi}_{\theta_{\text{old}}}$, $V_{\phi_{\text{old}}}$ are used for sampling CoT and computing $\hat{A}_t$. A clipped objective algorithm is employed for training [45].

Specifically, we encourage the model to explore different transformation paths using operators. We apply add-k smoothing to the distributions $\boldsymbol{\pi}_{\theta_{\text{old}}}$ for operator name tokens:

$$p'(a) = \frac{p(a) \times n + k}{n + |V_O| \times k} \tag{9}$$

where $p(a)$ denotes the probability of selecting action a before smoothing, n denotes the number of previously sampled training data, $|V_O|$ denotes the total number of possible operators corresponding to the vocabulary size, and k is the hyper-parameter for add-k smoothing. Therefore, in the early stages of sampling when n is 0, this strategy encourages uniform sampling of various operators to promote exploration. As training progresses and the number of samples increases, the probabilities gradually shift to align with on-policy sampling $\boldsymbol{\pi}_{\theta_{\text{old}}}$.

3.4 Reranking and Fallback at Inference Time

TabFormer does not always successfully transform complex tables. We propose a fallback mechanism: if a sample is transformed in violation of these requirements or has low expected rewards, we either regenerate a solution or maintain the original table without transformation.

A reward model assesses the accuracy of transformations followed by reasoning. We gather training data for the reward model by sampling 100 CoTs per validation sample using the ReFT model [12, 49, 50], assigning scores based on the relational normalization of transformed data and the ground truth alignment of the reasoning outputs. Initialized from the policy model, the reward model employs a linear classifier on the final token's hidden state [50]. This approach aligns with outcome-based reward models, effectively distinguishing good from bad solutions. At inference, if the first solution meets the relational normalization and achieves a "good" reward score above a pre-defined threshold $\mathcal{R}$, it is accepted as the final answer. Otherwise, a second solution is generated. If five generations are reached, the answer with the highest reward is selected as the reasoning output, and the original table is preserved without transformation.

4 Experiments

4.1 Baselines

Table Reasoning. For reasoning over hierarchical tables in HiTab and MultiHiertt, we select EEDP [46], a recent prompting strategy that elicits domain knowledge, extracts evidence, decomposes tasks, and predicts answers, and E5 [68], a code-augmented framework that combines self-explanation, code generation, and LLM reasoning to enhance accuracy. To ensure fair comparison using the full test set, we removed the test data filtering used in EEDP. For relational table reasoning, we use ReAcTable [66] and Chain-of-table [56], two strong methods that generate step-by-step code. Additionally, we include two leading LLMs on table reasoning tasks via instruction tuning, TableLlama [64] and StructLM [71], both re-finetuned in our experiments to incorporate held-out datasets for fair comparison. We maintain the same LLM version and table encoding and reasoning methods across all baselines.

Table Transformation. We employ NormTab[37] that normalizes tables by reorganizing and standardizing data values using LLMs, but its structural normalization includes only a single transposition operator, limiting its adaptability to hierarchical table structures.

TabFormer. For ablation, we implement the following variants:

w/o chaining operations. Generates relational tables in one step, omitting soft operations (Section 3.1).

w/o relational table generation via SQL. Refer to Section 3.1.

w/o final reasoning reward. Refer to Section 3.3.4.

w/o stepwise reasoning reward. Refer to Section 3.3.4.

w/o relational normalization reward. Refer to Section 3.3.3.

w/o ReFT. Eliminates ReFT, preserving only warm-up SFT.

w/o reranking and fallback. Refer to Section 3.4.

To assess the sensitivity of our method to different LLMs, we implement TabFormer and all baselines using three LLMs:

Llama (meta-Llama/Llama-3-8B): Meta Llama License.

Qwen (Qwen/Qwen2.5-7B-Instruct): Apache License.

Phi (microsoft/Phi-3-mini-128k-instruct): MIT License.

4.2 Implementation Details

We fix the *temperature* and *top_p* to 0 for all LLMs at inference time to ensure a fair comparison. Open-source LMs utilize DeepSpeed [41, 42] for distributed training and HuggingFace Accelerate [22] on 8 A100-80GB GPUs. During the warm-up stage of ReFT, we use the AdamW [34] optimizer with a learning rate of 3*e*-6 and a 0.1 warm-up ratio. The batch size is set to 12. The maximum length is set to 8,096. The number of epochs in the warm-up stage is 2. During the PPO stage of ReFT, the model is trained for 120 epochs with a learning rate of 3*e*-7. The λ, γ, α, and ϵ in PPO are set to 1, 0.95, 5, and 0.2, respectively. The number of updates per RL step is set to 2. The KL coefficient β is set to 0.05. k is set to 3,000 in add-k smoothing. The reward model for reranking is trained for 4 epochs using a batch size of 12 and a threshold $\mathcal{R}$ of 0.5.

Table 2: Comparison results for table QA and fact verification: all methods are evaluated in two settings. In the first, the semi-structured tables (T) before transformation serve as LLM input, while in the second, the relational tables (R) transformed via TabFormer (the Llama-3-8B-ReFT version) are used.

Reasoning method	LLM version	HiTab			MultiHiertt			WikiTQ			TabFact			AVG
		T	R	Δ	T	R	Δ	T	R	Δ	T	R	Δ	Δ
TableLlama [64]	Llama-2-7B-SFT	64.7	76.2	+11.5	49.2	61.5	+12.3	49.6	57.5	+7.9	82.8	86.7	+3.9	+8.9
StructLM [71]	CodeLlama-13B-SFT	56.5	73.6	+17.1	45.1	60.4	+15.3	53.2	60.5	+7.3	84.8	88.3	+3.5	+10.8
EEDP [46]	GPT-4	80.4	85.2	+4.8	65.9	71.1	+5.2	61.5	66.6	+5.1	85.6	87.9	+2.3	+4.3
EEDP [46]	GPT-3.5-turbo	34.8	54.4	+19.6	35.0	50.1	+15.1	44.0	50.2	+6.2	68.5	73.7	+5.2	+11.5
E5 [68]	GPT-4	83.4	87.5	+4.1	65.5	71.1	+5.6	65.0	68.0	+3.0	88.9	90.9	+2.0	+3.7
E5 [68]	GPT-3.5-turbo	36.6	56.9	+20.3	34.8	51.5	+16.7	47.6	54.2	+6.6	71.2	74.1	+2.9	+11.6
ReAcTable [67]	GPT-code-davinci-002	36.3	64.6	+28.3	32.3	56.7	+24.4	65.7	69.5	+3.8	83.3	85.8	+2.5	+14.8
ReAcTable [67]	GPT-3.5-turbo	28.5	54.3	+25.8	26.7	52.5	+25.8	52.6	58.5	+5.9	73.1	76.6	+3.5	+15.2
Chain-of-table [56]	GPT-3.5-turbo	29.8	57.4	+27.6	27.3	55.4	+28.1	59.8	66.4	+6.6	80.1	84.4	+4.3	+16.7
Chain-of-table [56]	Llama-2-17B	21.5	51.4	+29.9	20.8	46.6	+25.8	42.8	50.8	+8.0	67.3	72.5	+5.2	+17.2

4.3 Evaluation Metrics

Table Reasoning Following [6, 9, 60, 69], we use the accuracy metric, counting the number of examples where the system outputs the correct answer with Exact Match (EM), regardless of whether the answer is generated directly or produced by executing code.

4.4 Main Results

Table 2 presents the performance of table reasoning baselines on QA and FV tasks over input tables, both before and after transformation using TabFormer. Table 3 shows the comparison results of table transformation methods and their variants on reasoning tasks.

(1) Effectiveness of Transformation Followed by Reasoning: As shown in Table 2, across all four datasets, all baselines benefit significantly from proactive relational transformation by TabFormer, with accuracy gains ranging from 4% to 17%. Particularly for ReAcTable and Chain-of-table, which require executing SQL or Python code on relational(-like) tables, over 24% accuracy gains are observed on HiTab and MultiHiertt, which include rich hierarchical tables. Notably, even for E5 and EEDP, which are based on advanced GPT-4, our method based on Llama-3-8B improves accuracy by about 4%. This demonstrates that well-structured relational tables are easier for LMs of various scales to comprehend.

(2) Significant Benefits of ReFT over SFT: As shown in Table 2, well-established models like TableLlama and StructLM, fine-tuned via instruction tuning, benefit significantly from proactive table transformation, achieving over 9% accuracy gains. Additionally, Table 3 highlights substantial improvements (e.g., 13% for HiTab with Llama) achieved by TabFormer over direct SFT for QA, demonstrating the effectiveness of jointly optimizing transformation and reasoning with ReFT.

(3) TabFormer Substantially Outperforms Table Transformation Baselines: The reasoning task accuracy presented in Table 3 demonstrates that TabFormer significantly outperforms NormTab.

(4) ReFT Boosts the Performance of TabFormer: As shown in Table 3, removing ReFT from TabFormer results in big accuracy drops, as the warm-up stage does not use human-labeled data.

(5) Effectiveness of Chain-of-transformation Using Soft Operators: Step-by-step transformation with soft operators enhances ReFT's performance, particularly for hierarchical tables that require composite operations, as illustrated in Table 3.

(6) Benefits of Relational Normalization: Experimental results in Table 3 demonstrate that relational normalization consistently improves model performance across all datasets, with particularly notable gains on MultiHiertt, which frequently involves generating multiple tables. We extracted keywords of normalization from TabFormer's generation results across all datasets.

(7) QA Correctness Rewards Play a Crucial Role: Removing the final and stepwise QA rewards leads to accuracy drops from 11% to 25%. We believe QA rewards are essential for ensuring the completeness and accuracy of the transformed information.

(8) Effectiveness of Reranking and Fallback Mechanism: The observed accuracy gains indicate that reranking and fallback mechanisms effectively mitigate failed transformations and prevent error propagation to downstream tasks.

5 Limitations

In this work, we focus on table retrieval and reasoning within a single document. However, retrieving and reasoning across multiple documents remains challenging. We believe TabFormer can enhance advancements in RAG for large-scale databases, such as TAG and DBCopilot [2, 53], for complex tables.

Another challenge arises from large tables. While our datasets mainly contain small to medium-sized tables that current LLMs handle effectively, Large spreadsheet tables remain unaddressed. Recently, SpreadsheetLLM [16] introduced SheetEncoder to compress large tables. Exploring efficient transformation for large tables is a promising direction for future work.

6 Related Work

Table Transformation. Transforming complex tables into relational forms remains a significant challenge due to the diverse formatting styles and structures in real-world documents. Early

Table 3: Ablation results for input tables, both before and after relational data transformation, are presented for table question answering and fact verification. All methods (E5, Chain-of-table, NormTab, and TabFormer) are implemented using three LLMs for fair comparison: Llama-3-8B, Qwen-2.5-7B, and Phi-3-mini-3B. *Vanilla LLMs* refer to no fine-tuning on our datasets.

Transformation and reasoning method	HiTab			MultiHiertt			WikiTQ			TabFact		
	Llama	Qwen	Phi	Llama	Qwen	Phi	Llama	Qwen	Phi	Llama	Qwen	Phi
Vanilla LLMs for QA & fact verification												
Semi-Structured table (T) + E5	32.2	33.7	29.6	30.5	33.4	25.4	34.6	37.1	33.3	63.1	65.6	61.4
Semi-Structured table (T) + Chain-of-table	22.4	24.5	20.6	21.4	26.0	20.0	45.6	47.6	44.0	71.7	73.1	69.8
SFT LLMs directly for table QA & fact verification												
Semi-Structured table (T) + TableLlama [64]	67.4	69.0	62.0	52.5	52.6	49.8	55.0	56.6	52.6	82.9	84.3	82.0
Table-transformation baselines + Vanilla LLMs for QA & fact verification												
$\text{NormTab}_{\text{Target}}$ [37] (R)	33.2	34.0	31.5	30.6	34.0	29.9	48.7	49.3	46.0	63.6	63.2	60.9
ReFT LLMs for table transformation + Vanilla LLMs for QA & fact verification												
TabFormer (R) + Chain-of-table	55.5	57.6	50.7	49.0	49.6	45.6	53.5	57.0	51.9	76.0	76.6	73.2
ReFT LLMs for table transformation & QA & fact verification												
TabFormer (R)	80.6	81.1	75.5	66.0	67.3	62.9	66.4	67.8	63.3	87.3	88.7	86.2
w/o ReFT, only preserving warm-up SFT	-36.1	-36.6	-35.3	-26.5	-24.3	-24.2	-13.3	-14.6	-12.7	-21.4	-22.8	-23.8
w/o chaining operations	-6.4	-6.3	-5.7	-4.6	-3.1	-4.0	-1.8	-1.4	-2.2	-0.9	-0.1	-0.4
w/o relational table generation via SQL	-3.3	-2.6	-1.3	-4.6	-5.0	-4.8	-3.9	-3.6	-4.7	-1.4	-2.3	-2.0
w/o final reasoning reward	-24.8	-23.7	-24.9	-17.1	-17.8	-17.2	-13.0	-10.9	-11.3	-11.6	-11.7	-13.4
w/o stepwise reasoning reward	-5.4	-5.6	-4.4	-4.3	-2.9	-3.9	-1.5	-1.6	-1.0	-1.3	-0.2	-0.3
w/o relational normalization reward	-2.3	-3.0	-1.8	-3.0	-4.2	-3.2	-1.3	-0.7	-0.6	-0.4	-0.4	-0.1
w/o reranking and fallback	-4.7	-5.0	-4.0	-4.9	-3.8	-4.6	-3.0	-2.8	-2.7	-1.8	-1.5	-1.0

work, such as Harris and Gulwani [23], focused on replicating transformations from user-provided examples. Tools like FlashRelate [1] extended this by learning patterns directly from examples. More recent methods use classification to identify hierarchies and extract relational tuples from spreadsheets [7, 54]. AutoTables [29] pioneered synthesizing transformation datasets by converting relational tables into complex ones using predefined operators. Another method, NormTab [37], focuses on structural normalization but relies on a single transposition operator, limiting adaptability to general structures and ignoring multi-table relationships. To address these gaps, we propose TabFormer, introducing soft operators for transformation and supporting the generation of multiple tables through relational normalization.

Step-by-step Table Reasoning. The most relevant works include Chain-of-Table [55] and ReacTable [67]. Unlike these, our method optimizes both table transformation and reasoning within a single LLM call, leveraging our proposed soft operators that are tolerant to human formatting and structuring artifacts.

Reinforcement Learning. To our knowledge, this is the first work to propose reinforcement learning for table transformation and reasoning. Rewards, while crucial in reinforcement learning, are often difficult to obtain. Recent studies include LLM-as-a-Judge for verification [40, 70] and reward models like the Outcome Reward Model [59, 62] and Process Reward Model (PRM)[31], which provide dense, step-level rewards for reasoning[35, 52]. However, PRMs rely on costly step-level annotations [31], and automated methods like Monte Carlo Sampling or MCTS [35, 52] struggle to produce precise reward scores, limiting performance gains. We propose a relational normalization reward to encourage normalized outputs and final/stepwise reasoning correctness rewards to promote accuracy and reduce reward sparsity. We adopt PPO for optimization, as DPO and IPO are offline methods that cannot independently explore more CoT paths, limiting their performance to data from sub-optimal policies [49].

7 Conclusion

In this paper, we introduce TabFormer, a novel framework that leverages LLMs to normalize one or multiple semi-structured tables into fully relational formats and to enable downstream symbolic retrieval and reasoning. To effectively handle the various formatting artifacts present in tables, we design operators for structure transformation and value normalization to be "soft" by incorporating high-level semantic parameters. By combining supervised fine-tuning on synthetic examples with reinforcement learning guided by a relational normalization reward and a reasoning correctness reward, TabFormer effectively learns optimal transformation strategies without human-labeled data. Our extensive experiments on WTQ, HiTab, MultiHiertt, and TabFact demonstrate consistent accuracy improvements on strong baselines in both online and offline settings. Looking forward, we plan to explore more comprehensive relational normalization metrics and incorporate weak supervision from real user interactions.

8 Acknowledgments

This work was supported in part by the National Natural Science Foundation of China (Nos. U2336202, U21B2009).

References

[1] Daniel W Barowy, Sumit Gulwani, Ted Hart, and Benjamin Zorn. 2015. FlashRelate: extracting relational data from semi-structured spreadsheets using examples. *ACM SIGPLAN Notices* 50, 6 (2015), 218–228.

[2] Asim Biswal, Liana Patel, Siddarth Jha, Amog Kamsetty, Shu Liu, Joseph E Gonzalez, Carlos Guestrin, and Matei Zaharia. 2024. Text2sql is not enough: Unifying ai and databases with tag. *arXiv preprint arXiv:2408.14717* (2024).

[3] Michael J Cafarella, Alon Halevy, Daisy Zhe Wang, Eugene Wu, and Yang Zhang. 2008. Webtables: exploring the power of tables on the web. *Proceedings of the VLDB Endowment* 1, 1 (2008), 538–549.

[4] Peter Pin-Shan Chen. 1976. The entity-relationship model—toward a unified view of data. *ACM transactions on database systems (TODS)* 1, 1 (1976), 9–36.

[5] Wenhu Chen. 2022. Large language models are few (1)-shot table reasoners. *arXiv preprint arXiv:2210.06710* (2022).

[6] Wenhu Chen, Hongmin Wang, Jianshu Chen, Yunkai Zhang, Hong Wang, Shiyang Li, Xiyou Zhou, and William Yang Wang. 2019. Tabfact: A large-scale dataset for table-based fact verification. *arXiv preprint arXiv:1909.02164* (2019).

[7] Zhe Chen and Michael Cafarella. 2014. Integrating spreadsheet data via accurate and low-effort extraction. In *Proceedings of the 20th ACM SIGKDD international conference on Knowledge discovery and data mining*. 1126–1135.

[8] Zhoujun Cheng, Haoyu Dong, Ran Jia, Pengfei Wu, Shi Han, Fan Cheng, and Dongmei Zhang. 2022. FORTAP: Using Formulas for Numerical-Reasoning-Aware Table Pretraining. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 1150–1166.

[9] Zhoujun Cheng, Haoyu Dong, Zhiruo Wang, Ran Jia, Jiaqi Guo, Yan Gao, Shi Han, Jian-Guang Lou, and Dongmei Zhang. 2022. HiTab: A Hierarchical Table Dataset for Question Answering and Natural Language Generation. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 1094–1110.

[10] Zhoujun Cheng, Tianbao Xie, Peng Shi, Chengzu Li, Rahul Nadkarni, Yushi Hu, Caiming Xiong, Dragomir Radev, Mari Ostendorf, Luke Zettlemoyer, et al. 2022. Binding language models in symbolic languages. *arXiv preprint arXiv:2210.02875* (2022).

[11] Zhoujun Cheng, Tianbao Xie, Peng Shi, Chengzu Li, Rahul Nadkarni, Yushi Hu, Caiming Xiong, Dragomir Radev, Mari Ostendorf, Luke Zettlemoyer, Noah A. Smith, and Tao Yu. 2023. Binding Language Models in Symbolic Languages. *ICLR* (2023).

[12] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, et al. 2021. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168* (2021). https://arxiv.org/abs/2110.14168

[13] Edgar F Codd. 1970. A relational model of data for large shared data banks. *Commun. ACM* 13, 6 (1970), 377–387.

[14] Xiang Deng, Huan Sun, Alyssa Lees, You Wu, and Cong Yu. 2022. Turl: Table understanding through representation learning. *ACM SIGMOD Record* 51, 1 (2022), 33–40.

[15] Haoyu Dong, Shijie Liu, Shi Han, Zhouyu Fu, and Dongmei Zhang. 2019. TableSense: Spreadsheet table detection with convolutional neural networks. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 33. 69–76.

[16] Haoyu Dong, Jianbo Zhao, Yuzhang Tian, Junyu Xiong, Mengyu Zhou, Yun Lin, José Cambronero, Yeye He, Shi Han, and Dongmei Zhang. 2024. Encoding Spreadsheets for Large Language Models. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. 20728–20748.

[17] Wensheng Dou, Shi Han, Liang Xu, Dongmei Zhang, and Jun Wei. 2018. Expandable group identification in spreadsheets. In *Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering*. 498–508.

[18] Dawei Gao, Haibin Wang, Yaliang Li, Xiuyu Sun, Yichen Qian, Bolin Ding, and Jingren Zhou. [n. d.]. Text-to-SQL Empowered by Large Language Models: A Benchmark Evaluation. ([n. d.]).

[19] Dawei Gao, Haibin Wang, Yaliang Li, Xiuyu Sun, Yichen Qian, Bolin Ding, and Jingren Zhou. 2023. Text-to-SQL Empowered by Large Language Models: A Benchmark Evaluation. arXiv:2308.15363 [cs.DB]

[20] Jonas Gehring, Kunhao Zheng, Jade Copet, Vegard Mella, Taco Cohen, and Gabriel Synnaeve. [n. d.]. Rlef: Grounding code llms in execution feedback with reinforcement learning, 2024. *URL https://arxiv. org/abs/2410.02089* ([n. d.]).

[21] Majid Ghasemi Gol, Jay Pujara, and Pedro Szekely. 2019. Tabular cell classification using pre-trained cell embeddings. In *2019 IEEE International Conference on Data Mining (ICDM)*. IEEE, 230–239.

[22] Sylvain Gugger, Lysandre Debut, Thomas Wolf, Philipp Schmid, Zachary Mueller, Sourab Mangrulkar, Marc Sun, and Benjamin Bossan. 2022. Accelerate: Training and inference at scale made simple, efficient and adaptable. https://github.com/huggingface/accelerate.

[23] William R Harris and Sumit Gulwani. 2011. Spreadsheet table transformations from examples. *ACM SIGPLAN Notices* 46, 6 (2011), 317–328.

[24] Jonathan Herzig, Pawel Krzysztof Nowak, Thomas Mueller, Francesco Piccinno, and Julian Eisenschlos. 2020. TaPas: Weakly Supervised Table Parsing via Pretraining. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. 4320–4333.

[25] Deyi Ji, Lanyun Zhu, Siqi Gao, Peng Xu, Hongtao Lu, Jieping Ye, and Feng Zhao. 2024. Tree-of-Table: Unleashing the Power of LLMs for Enhanced Large-Scale Table Understanding. *arXiv preprint arXiv:2411.08516* (2024).

[26] Ralph Kimball and Margy Ross. 2013. *The data warehouse toolkit: The definitive guide to dimensional modeling*. John Wiley & Sons.

[27] Ralph Kimball and Margy Ross. 2013. *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling* (3rd ed.). Wiley.

[28] Solomon Kullback and Richard A Leibler. 1951. On information and sufficiency. *The annals of mathematical statistics* 22, 1 (1951), 79–86. https://www.jstor.org/stable/2236703

[29] Peng Li, Yeye He, Cong Yan, Yue Wang, and Surajit Chaudhuri. 2024. Auto-Tables: Relationalize Tables without Using Examples. *ACM SIGMOD Record* 53, 1 (2024), 76–85.

[30] Peng Li, Yeye He, Dror Yashar, Weiwei Cui, Song Ge, Haidong Zhang, Danielle Rifinski Fainman, Dongmei Zhang, and Surajit Chaudhuri. 2023. Table-gpt: Table-tuned gpt for diverse table tasks. *arXiv preprint arXiv:2310.09263* (2023).

[31] Hunter Lightman, Vineet Kosaraju, Yuri Burda, Harrison Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. 2024. Let's Verify Step by Step. In *The Twelfth International Conference on Learning Representations*. https://openreview.net/forum?id=v8L0pN6EOi

[32] Qian Liu, Bei Chen, Jiaqi Guo, Zeqi Lin, and Jian-guang Lou. 2021. Tapex: Table pre-training via learning a neural sql executor. *arXiv preprint arXiv:2107.07653* (2021).

[33] Tianyang Liu, Fei Wang, and Muhao Chen. 2023. Rethinking Tabular Data Understanding with Large Language Models. arXiv:2312.16702 [cs.CL]

[34] Ilya Loshchilov and Frank Hutter. 2017. Decoupled weight decay regularization. *arXiv preprint arXiv:1711.05101* (2017). https://arxiv.org/abs/1711.05101

[35] Liangchen Luo, Yinxiao Liu, Rosanne Liu, Samrat Phatale, Harsh Lara, Yunxuan Li, Lei Shu, Yun Zhu, Lei Meng, Jiao Sun, et al. 2024. Improve Mathematical Reasoning in Language Models by Automated Process Supervision. *arXiv preprint arXiv:2406.06592* (2024).

[36] Qingyang Mao, Qi Liu, Zhi Li, Mingyue Cheng, Zheng Zhang, and Rui Li. 2024. PoTable: Programming Standardly on Table-based Reasoning Like a Human Analyst. *arXiv preprint arXiv:2412.04272* (2024).

[37] Md Mahadi Hasan Nahid and Davood Rafiei. 2024. Normtab: Improving symbolic reasoning in llms through tabular data normalization. *arXiv preprint arXiv:2406.17961* (2024).

[38] Md Mahadi Hasan Nahid and Davood Rafiei. 2024. TabSQLify: Enhancing Reasoning Capabilities of LLMs Through Table Decomposition. *arXiv preprint arXiv:2404.10150* (2024).

[39] Panupong Pasupat and Percy Liang. 2015. Compositional semantic parsing on semi-structured tables. In *ACL*.

[40] Zhenting Qi, Mingyuan Ma, Jiahang Xu, Li Lyna Zhang, Fan Yang, and Mao Yang. 2024. Mutual Reasoning Makes Smaller LLMs Stronger Problem-Solvers. *arXiv preprint arXiv:2408.06195* (2024).

[41] Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. 2020. Zero: Memory optimizations toward training trillion parameter models. In *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*. https://ieeexplore.ieee.org/abstract/document/9355301/

[42] Jeff Rasley, Samyam Rajbhandari, Olatunji Ruwase, and Yuxiong He. 2020. Deepspeed: System optimizations enable training deep learning models with over 100 billion parameters. In *Proceedings of SIGKDD*. https://dl.acm.org/doi/abs/10.1145/3394486.3406703

[43] Martin Riedmiller, Roland Hafner, Thomas Lampe, Michael Neunert, Jonas Degrave, Tom Wiele, Vlad Mnih, Nicolas Heess, and Jost Tobias Springenberg. 2018. Learning by playing solving sparse reward tasks from scratch. In *Proceedings of ICML*. https://arxiv.org/pdf/1802.10567.pdf

[44] John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel. 2018. High-Dimensional Continuous Control Using Generalized Advantage Estimation. arXiv:1506.02438 [cs.LG]

[45] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347* (2017). https://arxiv.org/abs/1707.06347

[46] Pragya Srivastava, Manuj Malik, Vivek Gupta, Tanuja Ganu, and Dan Roth. 2024. Evaluating llms' mathematical reasoning in financial document question answering. In *Findings of the Association for Computational Linguistics ACL 2024*. 3853–3878.

[47] Zhao Tan, Xiping Liu, Qing Shu, Xi Li, Changxuan Wan, Dexi Liu, Qizhi Wan, and Guoqiong Liao. 2024. Enhancing Text-to-SQL Capabilities of Large Language Models through Tailored Promptings. In *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024)*. 6091–6109.

[48] Alexander Trott, Stephan Zheng, Caiming Xiong, and Richard Socher. 2019. Keeping your distance: Solving sparse reward tasks using self-balancing shaped rewards. In *Proceedings of NeurIPS*. https://proceedings.neurips.cc/paper_files/paper/2019/file/64c26b2a2dcf068c49894bd07e0e6389-Paper.pdf

[49] Luong Trung, Xinbo Zhang, Zhanming Jie, Peng Sun, Xiaoran Jin, and Hang Li. 2024. Reft: Reasoning with reinforced fine-tuning. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 7601–7614.

[50] Jonathan Uesato, Nate Kushman, Ramana Kumar, Francis Song, Noah Siegel, Lisa Wang, Antonia Creswell, Geoffrey Irving, and Irina Higgins. 2022. Solving math word problems with process-and outcome-based feedback. *arXiv preprint arXiv:2211.14275* (2022). https://arxiv.org/abs/2211.14275

[51] Gust Verbruggen, Vu Le, and Sumit Gulwani. 2021. Semantic programming by example with pre-trained models. *Proceedings of the ACM on Programming Languages* 5, OOPSLA (2021), 1–25.

[52] Peiyi Wang, Lei Li, Zhihong Shao, R. X. Xu, Damai Dai, Yifei Li, Deli Chen, Y. Wu, and Zhifang Sui. 2024. Math-Shepherd: Verify and Reinforce LLMs Step-by-step without Human Annotations. arXiv:2312.08935 [cs.AI]

[53] Tianshu Wang, Hongyu Lin, Xianpei Han, Le Sun, Xiaoyang Chen, Hao Wang, and Zhenyu Zeng. 2023. DBCopilot: Scaling Natural Language Querying to Massive Databases. *arXiv preprint arXiv:2312.03463* (2023).

[54] Zhiruo Wang, Haoyu Dong, Ran Jia, Jia Li, Zhiyi Fu, Shi Han, and Dongmei Zhang. 2021. TUTA: Tree-based Transformers for Generally Structured Table Pre-training. In *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining*. 1780–1790.

[55] Zilong Wang, Hao Zhang, Chun-Liang Li, Julian Martin Eisenschlos, Vincent Perot, Zifeng Wang, Lesly Miculicich, Yasuhisa Fujii, Jingbo Shang, Chen-Yu Lee, et al. [n. d.]. Chain-of-Table: Evolving Tables in the Reasoning Chain for Table Understanding. In *The Twelfth International Conference on Learning Representations*.

[56] Zilong Wang, Hao Zhang, Chun-Liang Li, Julian Martin Eisenschlos, Vincent Perot, Zifeng Wang, Lesly Miculicich, Yasuhisa Fujii, Jingbo Shang, Chen-Yu Lee, et al. 2024. Chain-of-table: Evolving tables in the reasoning chain for table understanding. *arXiv preprint arXiv:2401.04398* (2024).

[57] Yang Wu, Yao Wan, Hongyu Zhang, Yulei Sui, Wucai Wei, Wei Zhao, Guandong Xu, and Hai Jin. 2024. Automated Data Visualization from Natural Language via Large Language Models: An Exploratory Study. *Proceedings of the ACM on Management of Data* 2, 3 (2024), 1–28.

[58] Tianbao Xie, Chen Henry Wu, et al. 2022. UnifiedSKG: Unifying and Multi-Tasking Structured Knowledge Grounding with Text-to-Text Language Models. In *EMNLP*.

[59] An Yang, Beichen Zhang, Binyuan Hui, Bofei Gao, Bowen Yu, Chengpeng Li, Dayiheng Liu, Jianhong Tu, Jingren Zhou, Junyang Lin, et al. 2024. Qwen2. 5-math technical report: Toward mathematical expert model via self-improvement. *arXiv preprint arXiv:2409.12122* (2024).

[60] Yi Yang, Wen-tau Yih, and Christopher Meek. 2015. Wikiqa: A challenge dataset for open-domain question answering. In *Proceedings of the 2015 conference on empirical methods in natural language processing*. 2013–2018.

[61] Yunhu Ye, Binyuan Hui, Min Yang, Binhua Li, Fei Huang, and Yongbin Li. 2023. Large Language Models Are Versatile Decomposers: Decomposing Evidence and Questions for Table-Based Reasoning. In *Proc. of SIGIR*.

[62] Fei Yu, Anningzhe Gao, and Benyou Wang. 2023. Outcome-supervised verifiers for planning in mathematical reasoning. *arXiv preprint arXiv:2311.09724* (2023).

[63] Liangyu Zha, Junlin Zhou, Liyao Li, Rui Wang, Qingyi Huang, Saisai Yang, Jing Yuan, Changbao Su, Xiang Li, Aofeng Su, et al. 2023. Tablegpt: Towards unifying tables, nature language and commands into one gpt. *arXiv preprint arXiv:2307.08674* (2023).

[64] Tianshu Zhang, Xiang Yue, Yifei Li, and Huan Sun. 2024. TableLlama: Towards Open Large Generalist Models for Tables. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*. 6024–6044.

[65] Xiaokang Zhang, Jing Zhang, Zeyao Ma, Yang Li, Bohan Zhang, Guanlin Li, Zijun Yao, Kangli Xu, Jinchang Zhou, Daniel Zhang-Li, et al. 2024. TableLLM: Enabling Tabular Data Manipulation by LLMs in Real Office Usage Scenarios. *arXiv preprint arXiv:2403.19318* (2024).

[66] Yunjia Zhang, Jordan Henkel, Avrilia Floratou, Joyce Cahoon, Shaleen Deep, and Jignesh M Patel. [n. d.]. ReAcTable: Enhancing ReAct for Table Question Answering. ([n. d.]).

[67] Yunjia Zhang, Jordan Henkel, Avrilia Floratou, Joyce Cahoon, Shaleen Deep, and Jignesh M. Patel. 2023. ReAcTable: Enhancing ReAct for Table Question Answering. arXiv:2310.00815 [cs.DB]

[68] Zhehao Zhang, Yan Gao, and Jian-Guang Lou. 2024. E5: Zero-shot Hierarchical Table Analysis using Augmented LLMs via Explain, Extract, Execute, Exhibit and Extrapolate. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*. 1244–1258.

[69] Yilun Zhao, Yunxiang Li, Chenying Li, and Rui Zhang. 2022. MultiHiertt: Numerical Reasoning over Multi Hierarchical Tabular and Textual Data. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 6588–6600.

[70] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. 2023. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In *Thirty-seventh Conference on Neural Information Processing Systems Datasets and Benchmarks Track*.

[71] Alex Zhuang, Ge Zhang, Tianyu Zheng, Xinrun Du, Junjie Wang, Weiming Ren, Stephen W Huang, Jie Fu, Xiang Yue, and Wenhu Chen. 2024. StructLM: Towards Building Generalist Models for Structured Knowledge Grounding. *arXiv preprint arXiv:2402.16671* (2024).

[72] Daniel M Ziegler, Nisan Stiennon, Jeffrey Wu, Tom B Brown, Alec Radford, Dario Amodei, Paul Christiano, and Geoffrey Irving. 2019. Fine-tuning language models from human preferences. *arXiv preprint arXiv:1909.08593* (2019). https://arxiv.org/pdf/1909.08593.pdf